

# RICM CarBot

## **Goal:**

Create a modifiable 2 wheeled robot capable of currently 4 functions:

- Ability to be remotely controlled using any web browser
- Autonomous operation by collision avoidance
- Ability to follow a thick black line
- Ability to travel in a programmed pattern.

The robot is to be built by students from age 8 on up using standard mechanical components, screws, motors, plastic components. Students will also learn how to solder a custom-built printed circuit which will be the basis of expandability and provide support circuitry to control motors and other sensors. You can switch modes with a small push button located on the custom printed circuit board. An onboard green LED flashes to indicate the selected modes. You can change mode anytime.

## **Hardware:**

The “brains” of the robot car is a pre-programmed Raspberry Pi Pico W. The Pico was selected due to its ability to avoid program corruption when power is removed, and its size. RICM needed to have a board that the Pico could plug into that also had an onboard motor controller capable of driving wheel motors from 2.5 – 12 volts at 800ma of current per motor. No readily available boards were located that fit our needs, so RICM developed their own PCB. This custom PCB has mounting place in the middle for the Pico, (2) L9110 motor drivers, a power LED, a pushbutton. It also includes power pins for 3.3 V and 5 V as well as associated grounds to provide power to other devices such as sensors, lights etc.

The board uses GPIO pins for hardware interfacing, GPIO-14 for the pushbutton, GPIO-18-19 for the right motor and GPIO-20-21 for the left motor, for a total of 5 GPIO pins, leaving many more GPIO pins for expansion & experimentation. No external wires are needed to connect the motor drivers to the Pico as they are permanently assigned on the PCB.

On the Pico, there are also (3) analog to digital converters (ADC). The Pico uses these to convert analog signals or varying voltage, represented as numbers (0-4096) to be used for working with sensors that operate with analog voltages. These ADCs need a *reference voltage* to set the analog voltage against. Therefore, the custom PCB connects the regulated 3.3 V of the Pico to the reference voltage of the ADCs. so the ADCs can convert analog voltages from 0 to 3.3 V to a number between 0-4096! The reference voltage *can* be changed by cutting a bowtie looking PCB trace and supplying the ADCs with another voltage reference. The new voltage reference *must* be *very* stable, so it is not recommended this should be done, unless you know what you are doing.

The motors will operate from 1.5 to 6 V, thus making for easy & simple battery operation. The car uses (3) AA batteries producing 4.5 V to the board and motors. The Pico needs 1.8 to 5.5 V to operate, so this battery pack is ideal for both the Pico and the motors.

The custom PCB mounts to a 3D-printed adapter plate, that is raised up so battery pack can slide under it. Software is all open source and can be modified using standard Python language at will by the student, or used as-is. All hardware, except the custom motor driver board, is readily available, as off-the-shelf parts.

### **Software:**

With the small exception of HTML & JavaScript used for the *remote control* section, and *pattern upload* section, the software uses no special libraries and uses standard Python programming language. We chose Python because it is a good beginners programming language. All software is included with the robot and can be easily modified using the ThonnyThonny development environment. Thonny is a free software environment that is easily loaded onto any PC (Mac, Linux, Windows). Users should be familiar with this application and Python.

The robot has 4 modes it can operate in:

1. Remote control (default)
2. Autonomous collision
3. Programmed patterns
4. Line following

You can select the mode with the pushbutton located on the RICM motor control board. Press it once and wait for blinking light to turn solid, then go off. Press the button *X* times, where *X* is the mode you want the robot to be in. Once selected, the LED on the Pico will flash the mode it is in. If you select a mode that the hardware does not support, or external hardware is missing, the LED will flash rapidly. You can change modes anytime.

## OPERATION

When power is applied, the green LED on the RICM Driver board will light up indicating that power is applied. The green LED on the Pico will blink slowly indicating the default mode it is in.

To change the mode, press the push button on the RICM Driver board once. The Pico will respond with a solid light, then it will switch off. At this point, press the button to set the mode: press it 3 times for mode 3, once for mode 1 etc. The Pico will now flash the LED to indicate the current mode.

**NOTE:** the mode will reset to the default mode each time you apply power. This default mode can be set in the *globalv.py* module in the *lib/* directory using Thonny.

### Testing motor connections

The motor connections are the four center pins in the plastic connector at the rear of the car. Make *sure* the wires are connected on those four center pins or the motors will not work properly. (see Figure-1) M1 is for motor 1 and M2 is for motor 2.

Since these motors have no polarity, it's a simple task to ensure motors are plugged in correctly. Use the *remote* mode and move the joystick fully forward. Both wheels should spin in a forward direction. If not, simply reverse the wires going to the motor that is spinning in the wrong direction. Once you get the forward and backward motion correct. Then test left and right wheels for proper direction of the robot. If direction is wrong, simply swap both motor wires while maintaining their orientation. Once this configuration is set, it never has to be changed. If you unplug the motors and forget where they went, you will have to re-do this step.

|     |     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 3v3 | 3v3 | 3v3 | 3v3 | M1a | M2a | 5v  | 5v  | 5v  | 5v  |
| Gnd | Gnd | Gnd | Gnd | M1b | M2b | Gnd | Gnd | Gnd | Gnd |

Figure-1. Plastic connector pin assignments

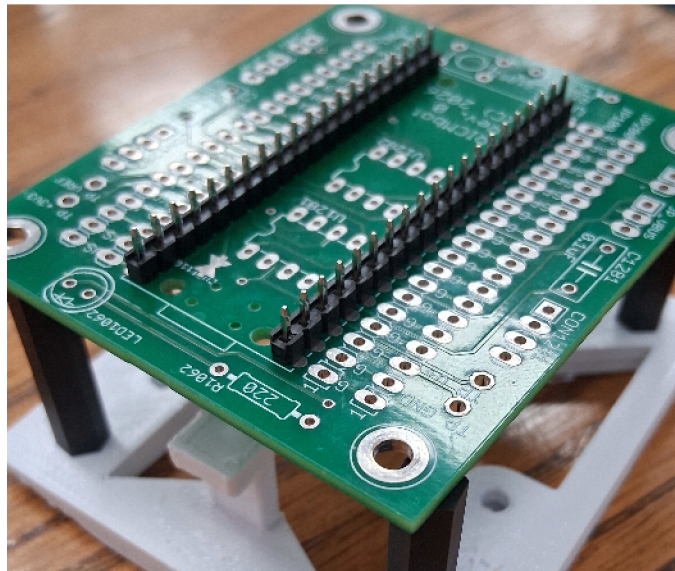
## ASSEMBLY

Make sure you have all the materials listed in the Bill of Materials first

### Soldering

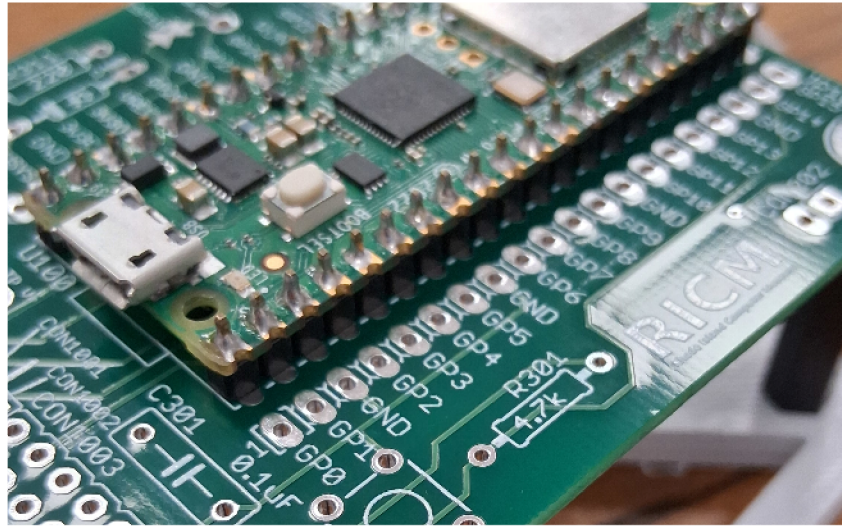
**NOTE: Check off each step as you proceed**

( ) Place the long pins of the male headers into the Printed Circuit Board (PCB) where the Raspberry Pi Pico would go. **DO NOT SOLDER THE PINS TO THE BOARD!!**

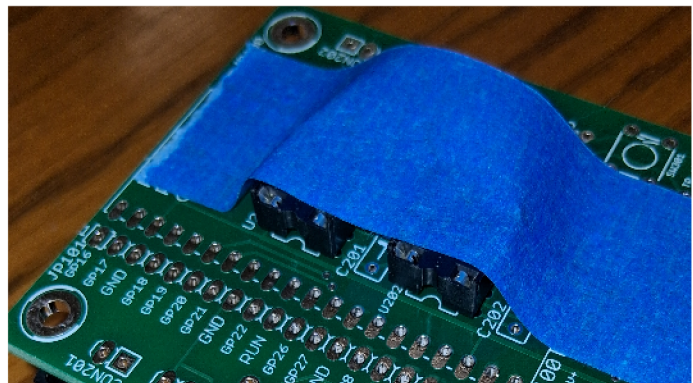
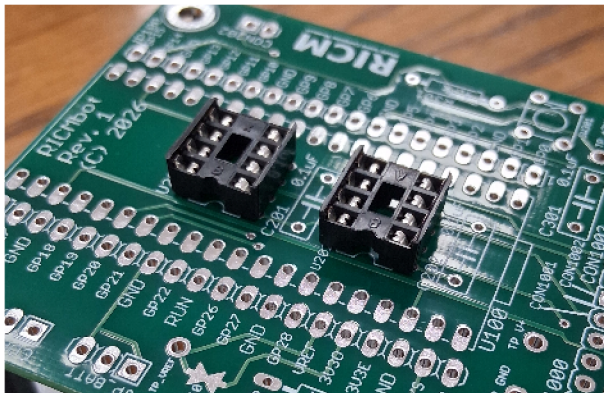




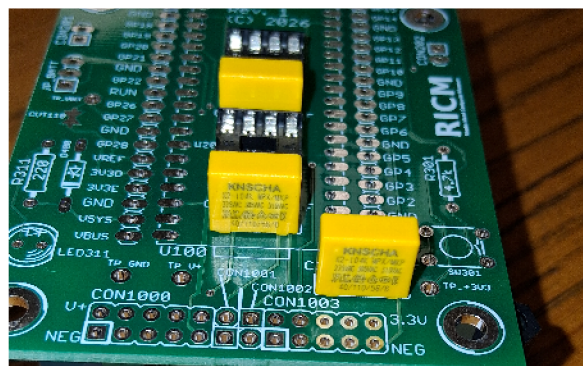
( ) 1. Place the Pico onto the short pins of the header. Solder the top and bottom pins on each side of the Pico. Make sure the headers are perfectly aligned and perpendicular with the Pico. At this point you can remove the Pico with the male headers and finish soldering the headers to the PCB. Inspect all the solder joints before continuing.



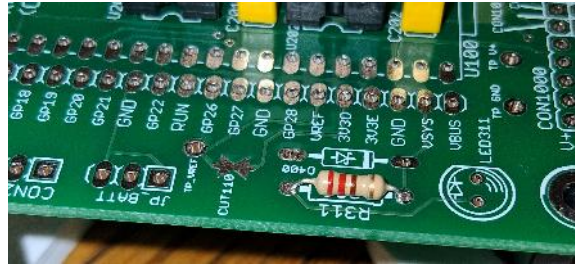
( ) 2. Solder the 8pin DIP sockets to the PCB, pay attention to the orientation of the sockets. You can use masking tape to hold the sockets in position if you need to.



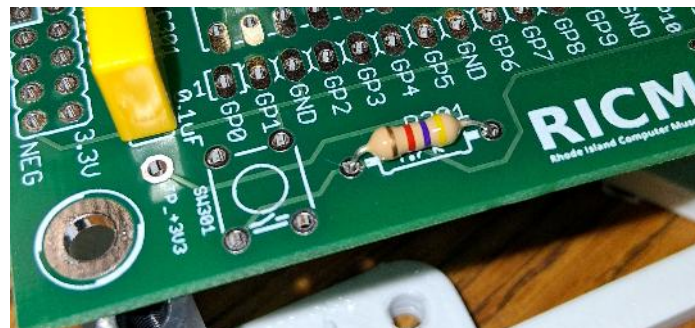
( ) 3. Solder the yellow capacitors into C201, C202, and C301. Make sure they are flush with the board. Clip off the excess leads from the bottom of the board.



( ) 4. Solder the 220 ohm (Red-Red-Brown) resistor into R311. Make sure it is flush with the board. Clip off the excess leads from the bottom of the board.



( ) 5. Solder the 4.7k ohm resistor (Yellow-Purple-Red) resistor into R301. Make sure it is flush with the board. Clip off the excess leads from the bottom of the board.



( ) 6. Solder the LED into LED311. All Diodes are polarity sensitive, so it's important that they are installed correctly. Failure to install correctly will result in the LED not working. Long leg is USUALLY the Anode or “+” side of LED. “+” side of LED is denoted on the left side of this symbol ▶ Clip off the excess leads from the bottom of the board.

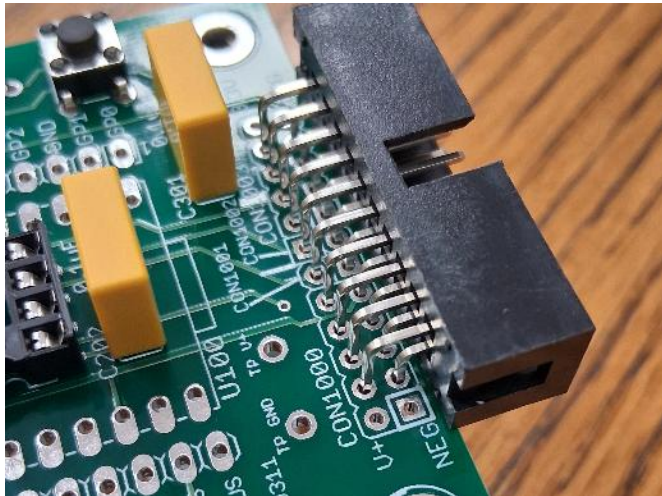




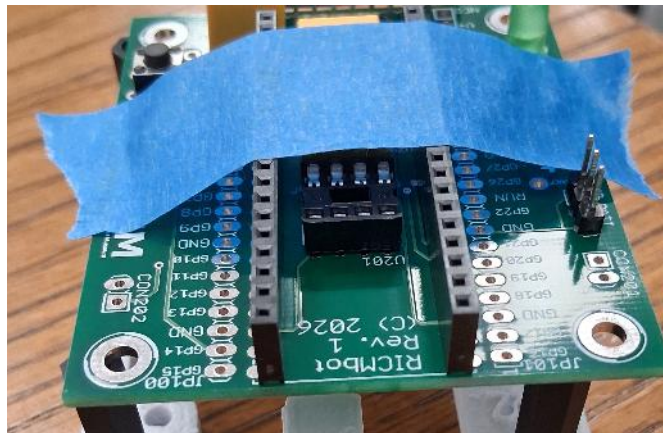
A close-up photograph of a green printed circuit board (PCB) populated with various electronic components. A black integrated circuit (microcontroller) is mounted in the center, with several yellow electrolytic capacitors placed around it. A small black push-button is visible on the left. Various surface-mount components, including resistors and smaller capacitors, are also present. The PCB features white silkscreen markings, including labels like 'GP2', 'GP3', 'GP4', 'GP5', 'GP6', 'GP7', 'GP8', and 'GP9'. The board is resting on a wooden surface.

A close-up photograph of a green printed circuit board (PCB) populated with various electronic components. The board features numerous silver-plated through-hole pads, many of which are populated with small, cylindrical surface-mount components. Labels such as GP16, GP17, GND, GP18, GP19, GP20, GP21, GND, GP22, RUN, GP26, GP27, GND, and GP28 are visible, indicating specific test points or component locations. A large circular hole is present in the upper left corner. A black integrated circuit (IC) is mounted on the right side, and a yellow component is partially visible on the far right. A white star-shaped marking is present near the bottom center, labeled CUT110. A small black component is labeled TP\_1REF. A white label with the text 'CON204' is visible on the left side. A black component is labeled U20.

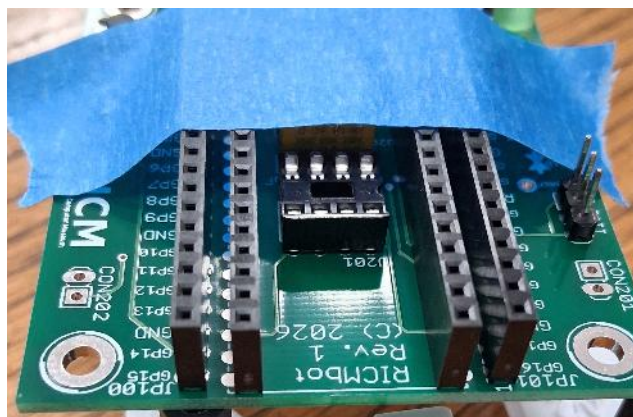
( ) 10. Solder the right-angle IDC connector. Leave the 1<sup>st</sup> and last position empty.



( ) 11. Solder the Pico female headers. Insert the headers onto the PCB, and hold them down with tape. Only solder the first and last pin at first to make sure they are aligned properly. After the two pins on each side are soldered in place and they are upright, finish soldering the headers in place.

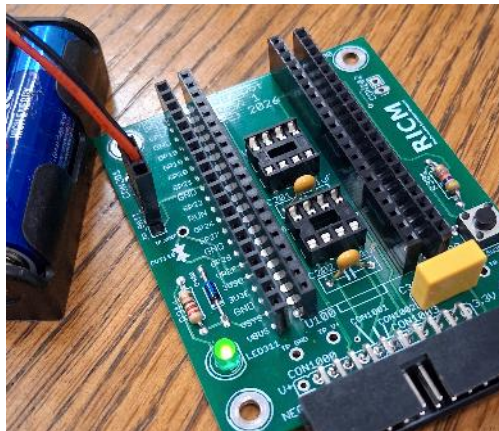


( ) 12. Solder the last two female headers next to the Pico headers. Use masking tape to hold them in place when you solder. Again, just solder the first and last pin and check for alignment *before* soldering the rest of the pins.



## **DO NOT INSTALL THE PICO OR MOTOR DRIVER CHIPS INTO THE SOCKETS**

At this point you may connect that battery pack to JP\_BATT. The LED should light. If the LED lights, you may continue. If not, remove the battery and check all soldering joints.



( ) 13. Strip off and tin about 1/8" of the insulation off each end of the female Dupont wires.

( ) 14. Remove the clear rubber hold down at the end of the motors and solder a wire to each of the motor terminals. Replace rubber hold down after soldering. Be careful not to overheat the motor connections.

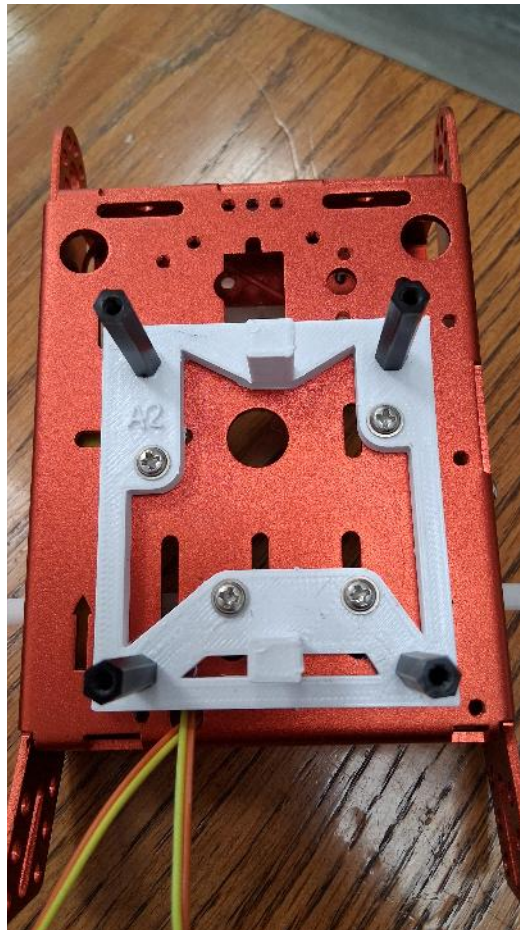


## Mechanical Assembly

( ) 15. Attach the 4 plastic posts to the top of the A2 plate using (4) 3mm screws.



( ) 16. Attach the plate to the rear of car chassis.



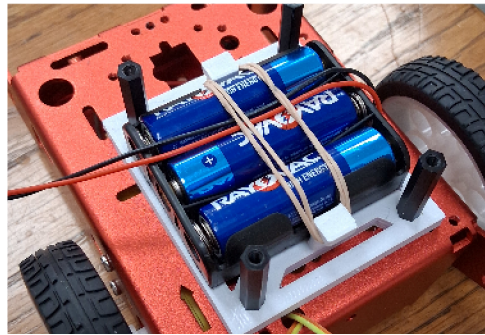
( ) 17. Attach each motor with long screws & nuts

( ) 18. Press the Wheels onto motor shafts

( ) 19. Attach the metal guide ball to the front of the chassis using two screws

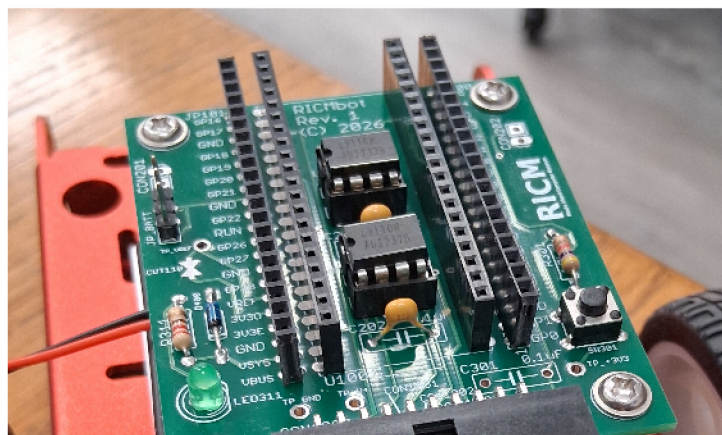
( ) 20. Guide the motor wires up through the chassis and through the small round hole at the back of the chassis.

( ) 21. Place the battery pack between the two guideposts and place a rubber band between the two posts to keep the battery in place.

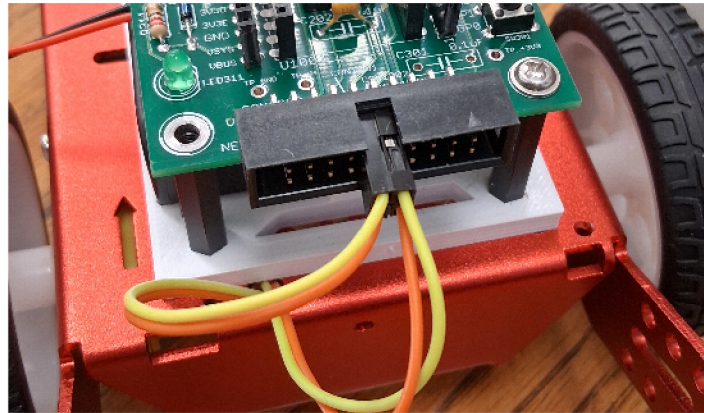


( ) 22. Attach the PCB to the four plastic posts with four 3mm screws.

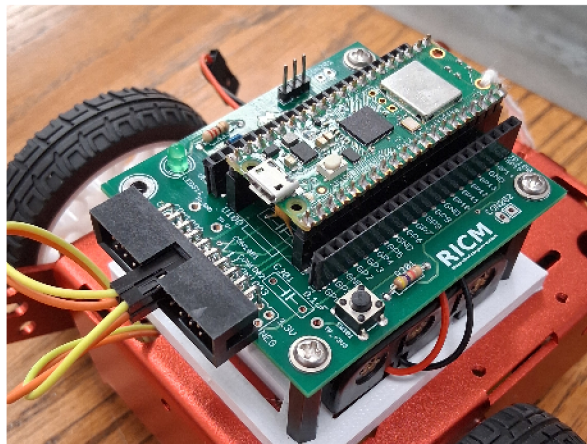
( ) 23. Install the two motor driver ICs pay *careful* attention to the notches in the ICs and sockets. Failure to install correctly will destroy the ICs. Make sure they are pressed all the way down. Inspect to determine whether any pins are not inserted fully.



( ) 24. Plug the wires from the motors to the center of the IDC connector at rear of car (see Figure-1)



( ) 24. Plug the Raspberry Pico into the female headers at the center of the PCB being careful to orient the Pico with the USB connection at the rear of the car chassis.



**DO NOT PLUG THE BATTERY IN YET!!!**



## Software install & configuration

### Install MicroPython (do this only once)

Go to Raspberry Pi website and follow instructions for installing MicroPython onto the Pico:

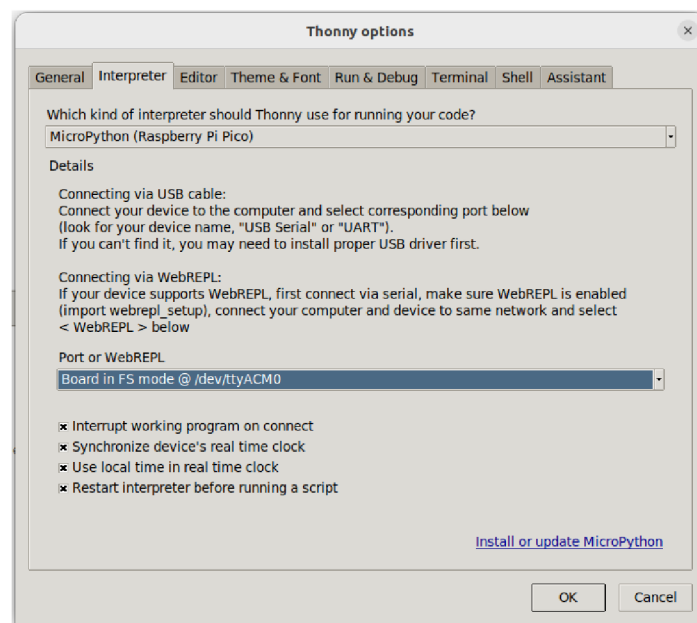
<https://www.raspberrypi.com/documentation/microcontrollers/micropython.html#what-is-micropython>

### Install ThonnyThonny

goto Go to <https://thonny.org/> to install Thonny for your computer

### Start ThonnyThonny

1. If you are on a Raspberry Pi, Thonny may be in Simple mode. Click **Switch to regular mode** on the right side of the toolbar, then exit and restart Thonny.
2. Click the **Tools** menu and select **Options**.
3. Under the **Interpreter** tab, ensure you make sure you have Micropython (Raspberry Pi Pico) selected, along with proper serial port.



4. Restart Thonny and you should see your Pico in the left column in the Files tab. If the Files tab is not visible, click the **View** menu and select **Files**.
5. Copy all of the supplied files to the Pico. The main.py goes in the root directory of the Pico, and ALL the .py files in the /lib directory, go into the Pico /lib directory
6. Copy *remote.html* & *pattern.html* to the root of the Pico

## Configuration:

1. Click the Lib directory on the Pico
2. Double-click the *globalv.py* file
3. Edit the robot name.
4. Edit the default mode (currently - REMOTE)
5. Save the file
6. Exit ThonnyThonny

Next: Unplug the USB cable

1. Plug the battery onto the PCB
2. The Pico should blink the default mode its in, repeatedly until changed via pushbutton

ENJOY!

## **THEORY OF OPERATION**

### **REMOTE CONTROL (Mode-1)**

In Mode-1, the default mode, the LED on the motor control board will flash once then a delay and repeat, indicating that it is in Mode-11. The name/password of the robot is stored in the *globalv.py* module. By default, the password is the same as the robot name, but can be changed in the *globalv* module.

In Mode-1, The robot will act as a WiFi access point, and the controlling device (i.e. laptop, iPad, phone, etc.) must be on this network. The SSID and password for the robot will be something like “Robotxxx” where xxx is a number like 001, 007 etc. This can be modified through software changes. The password for the access point is configured to be the same as the name you see when trying to connect. The XXX is written on the Pico, so it does not need to be remembered. Once connected, simply open a browser and type in “*robot.run*” in the URL line. Only one user at a time may connect with their browser to allow only user at a time to control the robot. Once connected, the user will be presented with an onscreen Joystick, allowing you to move the robot using a simple finger movement. Once you take your finger off the screen, the joystick will go to a neutral position and stop the robot. This is a fun way to control the robot.

### **Detailed operation:**

The *main.py* module calls the REMOTE\_MODE function in the *remote.py* module. This function should return a code of either a 0 (normal) or a negative number, indicating an error.

Once the REMOTE\_MODE function is called, it will set up an access point, DNS server, and web server in the *ws\_stuff.py* module. This module is responsible for all WiFi activity. When the services are activated, the function waits for a connection from a browser. Typically when a browser starts, it looks for a captive portal page. This has to be ignored via the DNS server to allow the user to type in a URL. The URL name that is used is stored in the *globalv.py* module. When the URL is typed in, the browser sends this to our DNS server, which resolves the name to the IP address of the Pico, where it connects to the HTTP server. The server will send the root document to the browser. This document is stored on the Pico root directory as *remote.html*. This web page must be limited to 6k in size. The web page also has some JavaScript (JS) associated with it. It is the job of the JS to handle communications from the joystick control to the *remote.py* module.

Since web traffic is VERY “chatty”, normal web conversation won’t be fast enough to handle the required speed. Therefore, the movement of the mouse was scaled to allow for +10 and -10 steps, both forward, backward, side to side. When the data is sent to the Pico it is scaled back up to +100 to -100 in all four directions, because the motor driver expects movement as a percentage. This greatly reduced the web traffic that would have occurred. Web traffic uses TCP, which requires each packet of data received to be

replied with an “ACK” before the next packet could be sent. Luckily the Pico TCP socket scheme handles this for us.

The “joystick” format is more intuitive to the user, so that’s the main reason why it was chosen. The user can close the browser anytime and the software will close the connection and wait again for another connection and start all over again. If the user presses the onboard push button on the PCB, the robot will shut down all services and allow the user to change the mode. This is a very simple mechanism as well.

The *remote.py* module requires the *ws\_stuff.py*, *globalv.py*, and *return\_codes.py* modules, in addition to the standard MicroPython *network*, *socket*, and *gc* libraries.

See the function listings for *remote.py* to see what each function does.



## **Autonomous collision avoidance (Mode-2)**

Mode-2, the LED on the Pico will continuously flash *twice*, then a delay and repeat while in this mode, indicating it is in Mode-2.

The robot will move along the ground at full speed, on its own, sensing any obstruction via an ultrasonic sensor, then as it gets close to an obstruction, it will slow down, giving the opportunity for the obstruction to move away, if not, the robot will stop, back away, and turn away from the obstruction, then continue on its journey in a forward motion. This mode will continue forever until changed via push button.

The Collision Avoidance (CA) module requires extra hardware and mounting material, such as a 3.3 V ultrasonic sensor, mounting adapter and wires to plug into the expansion headers. If this sensor is not present, the onboard LED will flash rapidly. During operation in this mode, the sensor sends out ultrasonic (40 kHz) sounds and listens for reflected sounds. The time difference from when the sound was sent, to the time it is sent back, indicates how far the object is. The intensity or “loudness” of reflected sound, indicates how large an object is. This is typically called a “ping” just the same way submarines detect objects in the water.

### **Pre-Programmed Patterns (Mode-3)**

Mode-3, the LED on the Pico will flash THREE times then a delay and repeat while in this mode, indicating it is in Mode-3.

This is another web-based module. Therefore upon selection of this mode, the *main* function will call *get\_pattern* in the *pattern.py* module. This function will set up an Access Point (AP), DNS server (DS), and a webserver (WS). These functions are in *ws\_stuff.py*. This module is also used with the *remote* module, so be careful if you modify it. Once the servers are active, the function waits for a web connection. Just like the remote mode, you first need to connect to the robot AP. Use the SSID name and password that is set in *globalv.py*. Without modification, the SSID and password are the same, typically used is RobotXXX, where XXX is a number like '007'. Your device may complain that there is no internet connection, which is fine for our purpose. Once your device has been connected to the AP, you need to open a browser to upload a pattern file to the robot.

When a browser is opened, it will attempt to search for a “captive portal”, which our DNS will ignore. You type into the URL line on the browser, our URL which is “*robot.run*”. This name is specified in the *globalv.py* module. If you modify this, do NOT use .com, .org .net etc. as your browser will want to connect via HTTPS instead of HTTP. HTTPS is way too complicated for the Pico to handle at this time. Upon successful connection, the browser will ask you for a file to upload using a standard file locating mechanism. Upon file selection, the browser will send the file a line at a time, checking each line for proper commands and format that is in the *cmd\_table.py* module. If anything is incorrect with a command or format, the import will fail with an error that needs to be corrected. If the line is correct, it is encoded in an internal pattern string.

Once the file has been successfully imported, the robot will follow this pattern endlessly, until a different mode is selected or power is removed. Once the pushbutton is pressed, the pattern data on the Pico will be cleared and memory reclaimed. The pattern module is very modular, allowing for new commands to be added, modified.

The user can write their own pattern for robot to follow.

Patterns can be created in a separate text file and name it whatever you like. ie. *pattern.txt*. The file is a plain text file with 1 command per line. Comments may be added to file of after a command. Comments begin with a “#” character, same as Python comments. It will ignore blank lines

Current pattern commands:

**SXXX** = Speed (0-100)

\*Once Speed setting is made, it stays that way until changed. Speed 0 = stop

**WXXX** = Wait in Milliseconds

**L** = Left turn

**R** = Right turn

**F** = Forward

**B** = Backward

**BBn** = Begin a block. (n is 1-10)

**EB** = End a block definition

**Exn** = Execute a block (n is the block number)

Sample file for a square:

```
# ----- Define a block -----
BB6      # begin block 6
S100     # set speed to 100%
F        # move fwd
W250     # wait 1000ms  (1 second)
S010     # Set speed to 10%
R        # right turn
W25      # wait 25ms. This is how long the wheels stay in this turn
S0       # stop
EB       # end this block - 6
# ----- end this block -----
# define square block
EX6      # turn 1
EX6      # turn 2
EX6      # turn 3
EX6      # turn 4
w2000    # wait 2 seconds after 4 turns
#----- Repeats pattern file endlessly
```

By modifying the *cmd\_table.py* it is possible to add more commands, such as turn on/off headlights, turn on/off turn/brake lights etc.

## **Line Following (Mode-4)**

### **Will be implimented in next release**

Mode-4, the LED on the custom printed circuit board will flash *four* times then a delay and repeat while in this mode, indicating it is in Mode-4. In this mode, the robot will follow a black line drawn on a light background.

## TECHNICAL DOCUMENTATION

**GENERAL:** As mentioned in the operation part of this manual, most of the software is written in standard MicroPython that runs in the Raspberry Pi Pico 2 W. No special libraries or dependencies are used. The software is modular by design, lending itself to easy modification. A program called Thonny is used to program the Pico. Thonny is a free program that runs on Apple, Windows, or Linux computers. The Pico is attached via USB cable to the computer.

Any setup variables such as the robot name, are in a file called *globalv.py* located in the *lib* directory on the Pico. There is also a **DEBUG** variable which can be set to **True**, to aid in debugging the code, through various print statements to the console.

Using the custom RICM motor driver board, an import in *main.py* must import *ricmboard*, as Platform, the *ricmboard* will import *L9110* as motor driver. This last import will let the software know how to actually control the motors.

Once a Platform and Motor have been established, the software will know how to control the robot. As per standard Pico operations, as soon as power is applied to the Pico, *main.py* will be executed.

Under Mode-1 (remote control) and Mode-3 (pre-programmed pattern control). WiFi and Web Servers are implemented. Using these two modes, some small JavaScript code is used to run on the users web browsers and is sent along with the web pages sent.



## **MAIN.PY**

This is the module that runs on power up. It sets up the base environment, and passes control to the default mode, which is set in *global.py*.

## **RICMMODE.PY**

This module is responsible for blinking the LED as a regular timer interval regardless of what the application is doing. It blinks the Pico LED with the current mode. It also has the ability to make the onboard LED Flash rapidly in the event something goes wrong with a mode setting. I.e setting the collision avoidance mode without the ultrasonic module being present, etc.

## **RICMBOARD.PY**

This module is unique to the RICM motor driver board. It also loads the *l9110.py* module, as this is the actual code that talks to the motors. This module also sets up the pushbutton interrupt handler. This pushbutton is unique to the RICM Board. The Interrupt handler just sets the *button\_pressed* variable located in *global.py*. This variable needs to be checked by any application and stop operation and return back to the *main.py* for proper mode handling. The actual mode switching is performed in the *ricmmode.py* module called from the *main.py* module.

## **L9110**

This is the module that drives the motors. It has two sections, 1 (initialization), 2 (actual driving of the motors). This module sets up Pulse Width Modulation (PWM), the GPIO Pins that are used, PWM Frequency, and Duty Cycle.

To make the motors move, the *drive* function is used. It takes two parameters, *d* and *t*. *d* is the differential, 0 to -100 for the left and 0 to 100 for the right. *t* is the throttle. 0 to 100 for a forward motion and 0 to -100 for reverse. For example, if you want to move forward, where each motor gets the power, set *d* to zero, and *t* to whatever speed you want from 0 to 100%. If you want reverse, again, set *d* to zero, and *t* to the speed you want to go backwards from 0 to -100. To stop the motors, set *d* to zero and *t* to zero.

## **REMOTE.PY**

This module is the heart of the remote control function. Upon successful web connection, the module will read *index.html* into memory and serve it to the user's web browser. *index.html* also contains some JavaScript that mimics a joystick control and as the user moves the "joystick" it will send updates to the Pico to make the motors move. When the user removes their finger from the joystick, it returns to its central position, and sends a motor stop command, causing the robot to stop any movement. Each pixel of movement is translated to 0-10 to minimize web traffic. The Pico can not handle fast, tiny, data transmission packets over WiFi, thus the joystick movement had to be scaled

down. Once the Pico receives the X/Y position data, it is converted to a 0-100% scale and also to a differential drive for each motor. This information is sent to the motor driver, in the case of the *ricmboard*, this will be the *l9110.py* motor driver as two parameters, differential and speed.

If the joystick is viewed as four quadrants, anything above the middle is 0-100, meaning motors will move in a forward direction. Anything below the middle will be 0 to -100, which means motors will spin backwards. Anything to the right will be 0 to 100 and result in more speed to the *left* motor, thus pushing the robot to the right. Anything to the left of center will be 0 to -100, resulting in more speed to the *right* motor, thus pushing it to the left.

Several *print()* statements are sprinkled throughout this module to aide in debugging if the variable “*debug*” is set to True in the *globalv* module

This module also frequently check the global variable *button\_pressed*. This means the user has pressed the button for a mode change. The module needs to clean up and exit if the button is pressed. This value will be True if the button was pressed. This module will stop the access point (AP), web server (WS), DNS server (DS) and stop all motors before exiting.